

SISTEMAS INTELIGENTES (PET1702)

Projeto 5 - Detecção de Intrusão utilizando Multi-Layer Perceptron

Componentes: **JEFFERSON BEZERRA DA GAMA (2025022550)**
LAVÍNIA DANTAS DE MESQUITA (2025022536)



I. OBJETIVO

Este projeto aborda o desafio da segurança de redes de computadores, especificamente a detecção de ataques como Negação de Serviço (DoS), varredura de portas (Probe) e acesso remoto não autorizado (R2L).

Como os atributos de um ataque muitas vezes se misturam ao tráfego legítimo de forma não-linear, uma rede Perceptron simples não é capaz de realizar a separação adequada. Para superar essa limitação, utilizaremos uma rede Multilayer Perceptron (MLP), que utiliza camadas ocultas e funções de ativação não-lineares para resolver problemas não linearmente separáveis. O treinamento será realizado através do algoritmo Backpropagation (retropropagação do erro).

O conjunto de dados utilizado é o NSL-KDD, uma versão refinada do histórico KDD Cup 1999, contendo amostras de conexões de rede com 41 atributos cada

II. PARTE TEÓRICA

As redes neurais artificiais são modelos computacionais inspirados no funcionamento do cérebro humano, capazes de aprender padrões complexos a partir de dados. O Multi-Layer Perceptron (MLP) é uma arquitetura de rede neural composta por múltiplas camadas de neurônios interconectados, incluindo uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída.

Diferentemente de modelos lineares, a MLP utiliza funções de ativação não-lineares, permitindo aprender relações complexas entre os atributos. O treinamento da rede é realizado por meio do algoritmo backpropagation, que ajusta os pesos utilizando gradiente descendente e minimização da função de perda.

Neste projeto, foi utilizada a função de perda BCEWithLogitsLoss, adequada para classificação binária, combinada ao otimizador Adam, responsável pela atualização eficiente dos pesos da rede. Além disso, foram empregadas técnicas como mini-batches, validação e early stopping, visando melhorar a estabilidade do treinamento e reduzir o overfitting.

III. PARTE PRÁTICA

Inicialmente, foi realizado o carregamento do dataset NSL-KDD, contendo informações sobre conexões de rede classificadas como normais ou ataques. Na etapa de pré-processamento, foram removidas variáveis irrelevantes, como "difficulty_level", e aplicadas técnicas de codificação categórica (*One-Hot Encoding*) para transformar atributos textuais em valores numéricos.

A variável alvo foi convertida em classificação binária, sendo 0 para conexões normais e 1 para ataques. Em seguida, os dados foram normalizados utilizando StandardScaler, garantindo que todas as variáveis apresentassem escalas compatíveis para treinamento da rede neural.

```
# Descartar a coluna 'difficulty_level'
df_train = df_train.drop('difficulty_level', axis=1)
df_test = df_test.drop('difficulty_level', axis=1)

# Binarizar o rótulo: normal -> 0; ataque -> 1
y_train = (df_train['attack_type'] != 'normal').astype(int)
y_test = (df_test['attack_type'] != 'normal').astype(int)

# Remover a coluna original de ataque dos atributos
X_train = df_train.drop('attack_type', axis=1)
X_test = df_test.drop('attack_type', axis=1)
```

```
# One-hot encoding nos atributos categóricos (protocol_type, service, flag)
X_train = pd.get_dummies(X_train, columns=['protocol_type', 'service', 'flag'])
X_test = pd.get_dummies(X_test, columns=['protocol_type', 'service', 'flag'])
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)

# Normalização (Z-score) usando apenas estatísticas do treino
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Fonte: Autoria Própria

O conjunto de dados foi dividido em treino, validação e teste. Posteriormente, os dados foram convertidos para tensores PyTorch e organizados em mini-batches utilizando DataLoader. A arquitetura da MLP foi definida empiricamente, utilizando camadas ocultas com função de ativação ReLU, buscando equilíbrio entre capacidade de aprendizado e custo computacional.

O treinamento foi realizado utilizando o otimizador Adam, função de perda BCEWithLogitsLoss e técnica de early stopping para evitar overfitting.

```
epochs = 50
train_losses = []
val_losses = []

best_val_loss = float('inf')
patience = 5
epochs_no_improve = 0
best_model_weights = None

for epoch in range(epochs):
    model.train()
    batch_losses = []

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer.zero_grad()
        predictions = model(X_batch)
        loss = criterion(predictions, y_batch)
        loss.backward()
        optimizer.step()

        batch_losses.append(loss.item())

    avg_train_loss = np.mean(batch_losses)
    train_losses.append(avg_train_loss)

    # Validação
    model.eval()
    with torch.no_grad():
```

```

X_val_device = X_val_tensor.to(device)
y_val_device = y_val_tensor.to(device)
val_preds = model(X_val_device)
val_loss = criterion(val_preds, y_val_device).item()
val_losses.append(val_loss)

print(f"Época [{epoch+1}/{epochs}] - Loss Treino: {avg_train_loss:.4f} | Loss
Validação: {val_loss:.4f}")

# Early Stopping
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_model_weights = model.state_dict()
    epochs_no_improve = 0
else:
    epochs_no_improve += 1
    if epochs_no_improve >= patience:
        print("Parada antecipada (Early Stopping) acionada.")
        break

```

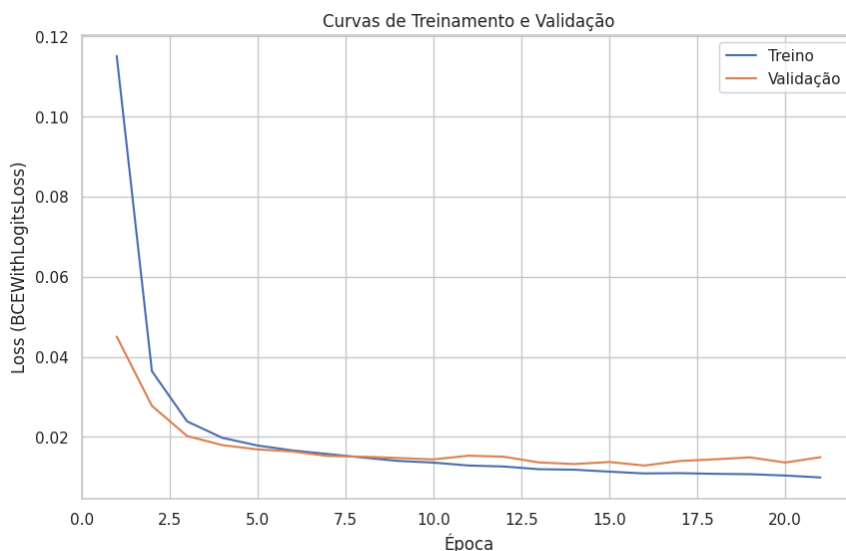
Fonte: Autoria Própria

IV. CONCLUSÕES

O pré-processamento foi fundamental para garantir o bom desempenho do modelo. A aplicação de One-Hot Encoding permitiu transformar atributos categóricos em representações numéricas adequadas para entrada na rede neural. A normalização dos dados contribuiu para estabilizar o treinamento, evitando que atributos com escalas maiores dominassem o processo de atualização dos pesos.

A arquitetura da MLP é adequada para este tipo de projeto pois busca equilíbrio entre desempenho e complexidade computacional. As camadas ocultas com funções de ativação ReLU permitem que a rede aprenda padrões não-lineares presentes nos dados de tráfego de rede. Já a escolha da quantidade de neurônios foi realizada empiricamente, considerando o desempenho observado durante o treinamento e validação.

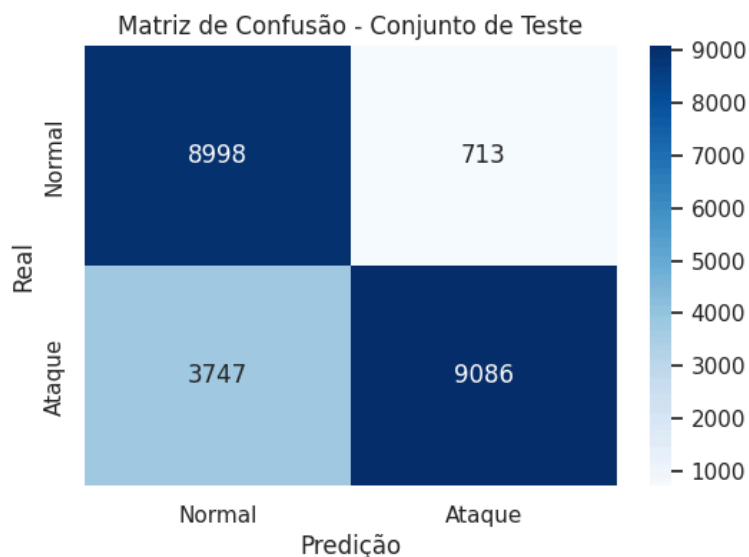
As curvas de treinamento e validação demonstraram comportamento estável ao longo das épocas, indicando aprendizado eficiente da rede neural.



Fonte: Autoria Própria

Observou-se redução acentuada da função de perda nas primeiras épocas, evidenciando rápida adaptação do modelo aos padrões presentes nos dados de tráfego de rede. Além disso, as curvas de treino e validação permaneceram próximas durante o treinamento, indicando boa capacidade de generalização e ausência significativa de overfitting.

A estabilização da função de perda após aproximadamente 10 épocas sugere que o modelo atingiu convergência, aprendendo representações relevantes sem memorizar excessivamente os dados de treino. A utilização da técnica de early stopping contribuiu para interromper o treinamento no momento adequado, preservando a melhor versão do modelo e reduzindo riscos de degradação da capacidade de generalização.



Fonte: Autoria Própria

A matriz de confusão demonstrou excelente capacidade da MLP em distinguir conexões normais de ataques. O modelo classificou corretamente 8998 conexões normais e 9086 ataques, evidenciando elevada capacidade de separação entre as classes. Os falsos positivos (713) representam conexões normais classificadas incorretamente como ataques, podendo gerar alarmes desnecessários no sistema. Já os falsos negativos (3747) representam ataques classificados incorretamente como conexões normais, sendo o erro mais crítico no contexto de sistemas IDS, pois permitem que ameaças passem despercebidas pela infraestrutura de segurança.

Apesar da presença de falsos negativos, o modelo apresentou desempenho geral bastante satisfatório, demonstrando elevada capacidade de aprendizado de padrões maliciosos em tráfego de rede. As métricas de avaliação confirmaram a eficiência do modelo, evidenciando boa capacidade de generalização e estabilidade durante a classificação.

A partir dos resultados obtidos, conclui-se que a rede neural MLP apresentou excelente desempenho na tarefa de detecção de intrusões, demonstrando alta capacidade de aprendizado e generalização. Entretanto, observou-se a presença de falsos negativos, representando ataques classificados incorretamente como tráfego normal. Esse tipo de erro é especialmente crítico em sistemas de segurança computacional, pois pode permitir que ameaças passem despercebidas pela infraestrutura monitorada.

V. CONSIDERAÇÕES FINAIS

No contexto de sistemas IDS (Intrusion Detection Systems), a redução de falsos negativos é extremamente importante, pois ataques não detectados podem comprometer a integridade, disponibilidade e confidencialidade dos sistemas monitorados.

Embora o modelo tenha apresentado excelente desempenho geral, a presença de falsos negativos indica que ainda existem padrões de ataque difíceis de identificar corretamente. Como possíveis melhorias, poderiam ser aplicadas técnicas de balanceamento de classes, ajuste do limiar de decisão ou arquiteturas neurais mais profundas, visando aumentar ainda mais a capacidade de detecção de ameaças.

REFERÊNCIAS

NSL-KDD Dataset. University of New Brunswick. Disponível em: <<https://www.unb.ca/cic/datasets/nsl.html>>. Acesso em: 10 mai. 2026.

HASSAN06. **NSL-KDD Dataset.** Kaggle, 2018. Disponível em: <<https://www.kaggle.com/datasets/hassan06/nslkdd>>. Acesso em: 11 mai. 2026.

SCIKIT-LEARN. **Preprocessing Data.** Disponível em: <<https://scikit-learn.org/stable/modules/preprocessing.html>>. Acesso em: 27 abr. 2026.

SCIKIT-LEARN. **Classification Metrics.** Disponível em: <https://scikit-learn.org/stable/modules/model_evaluation.html>. Acesso em: 27 abr. 2026.

SOUZA, Pedro Thiago Valério de. **Intelligent Systems - Slides.** GitHub, 2026. Disponível em: <<https://github.com/pedrothiag/intelligent-systems/tree/main/slides>>. Acesso em: 1 mai. 2026.